Core Language：候选 Trace 与同步公理

1　程序标注与事件解释

程序 $P = \mathsf{run}\{C_1 \parallel \cdots \parallel C_m\}$ 采用 `core-language-syntax.tex` 的语法，其中 $m \geq 1$。值域及元变量的范围为

$$\mathsf{Bool} = \{\mathsf{true}, \mathsf{false}\}, \qquad \mathsf{Val} = \mathsf{Int} \uplus \mathsf{Float} \uplus \mathsf{Bool} \uplus \mathsf{EventId} \uplus \mathsf{Pipe},$$
$$\nu, \nu_1, \nu_2, \ell \in \mathsf{Val}, \qquad \bar{\nu} \in \mathsf{Val}^*, \qquad p, q \in \mathsf{Pipe}, \quad b \in \mathsf{Bool}, \quad \varepsilon \in \mathsf{EventId},$$
$$\mathsf{Op} = \mathbb{N}_{>0} \times \mathbb{N}_{>0}, \qquad \mathsf{thread}(j, r) = j.$$

$\uplus$ 是带类别标记的不交并，注入记号省略。地址和锁由值指代；ℓ 用作锁标识。$\langle\rangle$ 表示空序列，uv 表示序列连接。

对每个分支 C_j，给其产生事件的语法位置分配互异标注 $o = (j, r) \in \mathsf{Op}$。用下标 o 表示该标注，例如 $x := \mathsf{read}_o[p](v); C$。别名绑定 $\mathsf{let}\ x = v\ \mathsf{in}\ C$ 不带事件标注。$\mathsf{head}(o)$ 表示位置 o 的命令头，取值为

$$\mathsf{read}(p),\ \mathsf{write}(p),\ \mathsf{compute}(p),\ \mathsf{setflag}(p, q),\ \mathsf{waitflag}(p, q),$$
$$\mathsf{lock},\ \mathsf{unlock},\ \mathsf{if},\ \mathsf{return}.$$

事件解释域定义为

$$K = \{(o, \alpha, \eta) \mid o \in \mathsf{Op},\ (\alpha, \eta) \in L\}, \qquad \boxed{\kappa : \mathsf{EventId} \longrightarrow K.}$$

L 是下表所有模式按上述元变量范围实例化所得二元组的集合。() 表示空载荷。

命令	发起事件的 (α, η)	完成事件的 (α, η)
read	$(\mathsf{read_start}, (p, \nu))$	$(\mathsf{read_end}, \nu)$
write	$(\mathsf{write_start}, (p, \nu_1, \nu_2))$	$(\mathsf{write_end}, ())$
compute	$(\mathsf{compute_start}, (p, \bar{\nu}))$	$(\mathsf{compute_end}, \nu)$
setflag	$(\mathsf{set_flag_start}, (p, q, \varepsilon))$	$(\mathsf{set_flag_end}, ())$
waitflag	$(\mathsf{wait_flag_start}, (p, q, \varepsilon))$	$(\mathsf{wait_flag_end}, ())$
lock	$(\mathsf{lock_start}, \ell)$	$(\mathsf{lock_end}, ())$
unlock	(unlock, ℓ)	同一事件
if	(branch, b)	同一事件
return	$(\mathsf{return}, \bar{\nu})$	同一事件

对 $e \in \mathsf{EventId}$，定义

$$(\mathsf{owner}(e), \mathsf{kind}(e), \mathsf{pay}(e)) = \kappa(e).$$

每次事件发生使用一个独立的 EventId 符号。同步操作数 ε 保存在载荷中，可以在多次同步中重复使用。

2 从程序生成候选 Trace

2.1 候选事件集合

令 $\rho : \mathsf{Var} \rightharpoonup \mathsf{Val}$ 为候选取值环境。定义

$$\rho(c) = c \quad (c \in \mathsf{Val}), \qquad \rho(\langle v_1, \ldots, v_k \rangle) = \langle \rho(v_1), \ldots, \rho(v_k) \rangle,$$

$$\rho[x \mapsto \nu](y) = \begin{cases} \nu, & y = x, \\ \rho(y), & y \neq x. \end{cases}$$

纯计算由参数关系 $\mathsf{Comp}_p \subseteq \mathsf{Val}^* \times \mathsf{Val}$ 给出，满足

$$\forall p \in \mathsf{Pipe}.\ \forall \bar{\nu} \in \mathsf{Val}^*.\ \exists \nu \in \mathsf{Val}.\ (\bar{\nu}, \nu) \in \mathsf{Comp}_p.$$

$\mathcal{E}_\rho(C)$ 表示由 C 生成的有限候选事件集合的集合。绑定、读取和计算的生成规则为

$$\mathcal{E}_\rho(\mathsf{let}\ x = v\ \mathsf{in}\ C) = \mathcal{E}_{\rho[x \mapsto \rho(v)]}(C),$$

$$\begin{aligned} \mathcal{E}_\rho(x := \mathsf{read}_o[p](v); C) = {} & \{\varnothing\} \cup \\ & \{\{(o, \mathsf{read_start}, (p, \rho(v))), (o, \mathsf{read_end}, \nu)\} \cup E \\ & \quad \mid \nu \in \mathsf{Val},\ E \in \mathcal{E}_{\rho[x \mapsto \nu]}(C)\}, \end{aligned}$$

$$\begin{aligned} \mathcal{E}_\rho(\mathsf{let}\ x = \mathsf{compute}_o[p](\bar{v})\ \mathsf{in}\ C) = {} & \{\varnothing\} \cup \\ & \{\{(o, \mathsf{compute_start}, (p, \rho(\bar{v}))), (o, \mathsf{compute_end}, \nu)\} \cup E \\ & \quad \mid (\rho(\bar{v}), \nu) \in \mathsf{Comp}_p,\ E \in \mathcal{E}_{\rho[x \mapsto \nu]}(C)\}. \end{aligned}$$

读取结果在候选生成中遍历 Val。ρ 记录候选中的取值关联；对 x 的绑定不增加事件之间的顺序关系。

用 $J_o; C$ 表示 write、setflag、waitflag、lock 或 unlock 命令及其后续程序。单条命令的事件集合 $B_\rho(J_o)$ 定义如下。

J_o	$B_\rho(J_o)$
$\mathsf{write}_o[p](v_1, v_2)$	$\{(o, \mathsf{write_start}, (p, \rho(v_1), \rho(v_2))), (o, \mathsf{write_end}, ())\}$
$\mathsf{setflag}_o[p, q](v)$	$\{(o, \mathsf{set_flag_start}, (p, q, \rho(v))), (o, \mathsf{set_flag_end}, ())\}$
$\mathsf{waitflag}_o[p, q](v)$	$\{(o, \mathsf{wait_flag_start}, (p, q, \rho(v))), (o, \mathsf{wait_flag_end}, ())\}$
$\mathsf{lock}_o(v)$	$\{(o, \mathsf{lock_start}, \rho(v)), (o, \mathsf{lock_end}, ())\}$
$\mathsf{unlock}_o(v)$	$\{(o, \mathsf{unlock}, \rho(v))\}$

生成规则为

$$\mathcal{E}_\rho(J_o; C) = \{\varnothing\} \cup \{B_\rho(J_o) \cup E \mid B_\rho(J_o) \subseteq K,\ E \in \mathcal{E}_\rho(C)\}.$$

条件分支和返回的生成规则为

$$\begin{aligned} & \mathcal{E}_\rho(\mathsf{if}_o\ v\ \mathsf{then}\ \{C_1\}\ \mathsf{else}\ \{C_2\}) \\ & \quad = \{\varnothing\} \cup \{\{(o, \mathsf{branch}, b)\} \cup E \mid b = \rho(v) \in \mathsf{Bool},\ E \in \mathcal{E}_\rho(C_b)\}, \\ & \qquad C_{\mathsf{true}} = C_1, \qquad C_{\mathsf{false}} = C_2, \end{aligned}$$

$$\mathcal{E}_\rho(\mathsf{return}_o(\bar{v})) = \{\varnothing, \{(o, \mathsf{return}, \rho(\bar{v}))\}\}.$$

各规则中的 $\varnothing$ 允许在当前位置停止展开。整个程序生成的候选事件集合为

$$\mathcal{E}(P) = \left\{ \bigcup_{j=1}^{m} E_j \;\middle|\; \forall j \in \{1, \ldots, m\}.\ E_j \in \mathcal{E}_\varnothing(C_j) \right\}.$$

2.2 事件排列与候选 Trace

对有限集合 X，定义其全部排列和前缀：

$$\mathsf{Perm}(X) = \{\langle a_1, \ldots, a_n\rangle \mid n = |X|,\ \{a_1, \ldots, a_n\} = X,\ \forall i \neq j.\ a_i \neq a_j\},$$
$$u \sqsubseteq v \iff \exists w.\ v = uw, \qquad \mathsf{Pref}(L) = \{u \mid \exists v \in L.\ u \sqsubseteq v\}.$$

将 κ 按元素扩展到序列：

$$\kappa(e_1 \cdots e_n) = \langle \kappa(e_1), \ldots, \kappa(e_n)\rangle.$$

对 $t \in \mathsf{EventId}^*$ 和 $\kappa : \mathsf{EventId} \to K$，候选 trace 的集合定义为

$$\boxed{\mathsf{Cand}(P) = \{(t, \kappa) \mid \exists E \in \mathcal{E}(P).\ \kappa(t) \in \mathsf{Pref}(\mathsf{Perm}(E))\}.}$$

候选生成保留语法位置、分支选择、操作数和结果的关联。排列中的事件记录互异，因而 trace 中的 EventId 标识也互异。排列的前缀包含尚未发起、尚未完成及不同命令交错的情形。

3 事件顺序与同步公理

以下固定 $(t, \kappa) \in \mathsf{Cand}(P)$，其中 $t = e_1 \cdots e_n$。事件集合和顺序定义为

$$\mathsf{ids}(t) = \{e_1, \ldots, e_n\}, \qquad e_i <_t e_j \iff i < j.$$

对 $A, B \subseteq \mathsf{ids}(t)$，定义

$$A \lhd_t B \quad \iff \quad \forall b \in B.\ \exists a \in A.\ a <_t b.$$

当 B 非空时，$A \lhd_t B$ 要求 A 中具有更早的事件。当 B 为空时，该条件成立。

命令 o 的发起、完成种类 $\sigma(o), \delta(o)$ 由第一个事件表的对应列给出。unlock、if、return 的两种种类相同。定义

$$S_t(o) = \{e \in \mathsf{ids}(t) \mid \mathsf{owner}(e) = o \wedge \mathsf{kind}(e) = \sigma(o)\},$$
$$D_t(o) = \{e \in \mathsf{ids}(t) \mid \mathsf{owner}(e) = o \wedge \mathsf{kind}(e) = \delta(o)\}.$$

候选生成保证 $S_t(o), D_t(o)$ 各自至多包含一个事件。

3.1 程序顺序

用 $Q_o \cdot C$ 统一表示带后续程序的事件命令，包括 read、compute 和 $J_o; C$。命令标识集合和程序顺序关系通过以下递归等式定义：

$$\mathsf{Op}(\mathsf{return}_o(\bar{v})) = \{o\}, \qquad \mathsf{po}(\mathsf{return}_o(\bar{v})) = \varnothing,$$
$$\mathsf{Op}(\mathsf{let}\ x = v\ \mathsf{in}\ C) = \mathsf{Op}(C), \qquad \mathsf{po}(\mathsf{let}\ x = v\ \mathsf{in}\ C) = \mathsf{po}(C),$$
$$\mathsf{Op}(Q_o \cdot C) = \{o\} \cup \mathsf{Op}(C), \qquad \mathsf{po}(Q_o \cdot C) = (\{o\} \times \mathsf{Op}(C)) \cup \mathsf{po}(C).$$

对 $I = \mathsf{if}_o\ v\ \mathsf{then}\ \{C_1\}\ \mathsf{else}\ \{C_2\}$，定义

$$\mathsf{Op}(I) = \{o\} \cup \mathsf{Op}(C_1) \cup \mathsf{Op}(C_2),$$
$$\mathsf{po}(I) = (\{o\} \times (\mathsf{Op}(C_1) \cup \mathsf{Op}(C_2))) \cup \mathsf{po}(C_1) \cup \mathsf{po}(C_2),$$
$$\mathsf{Op}(P) = \bigcup_{j=1}^{m} \mathsf{Op}(C_j), \qquad \mathsf{po}(P) = \bigcup_{j=1}^{m} \mathsf{po}(C_j).$$

继续条件对应的事件集合定义为

$$G_t(o) = \begin{cases} S_t(o), & \mathsf{head}(o) \in \{\mathsf{read}(p), \mathsf{write}(p), \mathsf{setflag}(p, q) \mid p, q \in \mathsf{Pipe}\}, \\ D_t(o), & \text{其他情形.} \end{cases}$$

A1：发起先于完成。

$$\forall o \in \mathsf{Op}(P).\ \sigma(o) \neq \delta(o) \Rightarrow S_t(o) \lhd_t D_t(o).$$

A2：程序顺序。

$$\forall (o,u) \in \mathsf{po}(P).\ S_t(o) \lhd_t S_t(u)\ \wedge\ G_t(o) \lhd_t S_t(u).$$

read、write、setflag 的继续条件对应发起事件；compute、waitflag、lock 的继续条件对应完成事件。

3.2 pipeline 与 flag

pipeline 归属和同一 pipeline 上的发起顺序定义为

$$\mathsf{pipe}(o) = \begin{cases} p, & \mathsf{head}(o) \in \{\mathsf{read}(p), \mathsf{write}(p), \mathsf{compute}(p), \mathsf{setflag}(p,q)\}, \\ q, & \mathsf{head}(o) = \mathsf{waitflag}(p,q), \\ \bot, & \mathsf{head}(o) \in \{\mathsf{lock}, \mathsf{unlock}, \mathsf{if}, \mathsf{return}\}, \end{cases}$$

$$o \prec_{p,t} u \iff \mathsf{pipe}(o) = \mathsf{pipe}(u) = p \wedge \exists a \in S_t(o), b \in S_t(u).\ a <_t b.$$

$\bot \notin \mathsf{Pipe}$ 表示没有 pipeline 归属。

A3：设置完成等待源 pipeline 的前序操作完成。

$$\boxed{\forall f, o \in \mathsf{Op}(P).\ \forall p, q \in \mathsf{Pipe}.\quad \mathsf{head}(f) = \mathsf{setflag}(p,q) \wedge o \prec_{p,t} f \Rightarrow D_t(o) \lhd_t D_t(f).}$$

前序范围由 set_flag_start 的位置确定，包含该源 pipeline 上此前发起的全部操作。

对 $E \subseteq \mathsf{ids}(t)$ 和字母映射 $\lambda : E \to \Sigma$，定义有序投影

$$\mathsf{proj}_t(E, \lambda) = \langle \lambda(e_{i_1}), \ldots, \lambda(e_{i_k}) \rangle,$$
$$i_1 < \cdots < i_k, \qquad \{i_1, \ldots, i_k\} = \{i \in \{1, \ldots, n\} \mid e_i \in E\}.$$

Σ 表示相应投影使用的字母集合；$k = 0$ 时投影为空序列。对同步键 $h \in \mathsf{Pipe} \times \mathsf{Pipe} \times \mathsf{EventId}$，定义

$$E_h(t) = \{e \in \mathsf{ids}(t) \mid \exists o \in \mathsf{Op}(P), s \in S_t(o).\ e \in D_t(o) \wedge \mathsf{pay}(s) = h \wedge \mathsf{kind}(e) \in \{\mathsf{set_flag_end}, \mathsf{wait_flag_end}\}\},$$

$$\lambda_h(e) = \begin{cases} S, & \mathsf{kind}(e) = \mathsf{set_flag_end}, \\ W, & \mathsf{kind}(e) = \mathsf{wait_flag_end}, \end{cases}$$

$$\pi_h(t) = \mathsf{proj}_t(E_h(t), \lambda_h).$$

S, W 是表示设置完成、等待完成的两个不同字母。

A4：匹配的设置完成与等待完成交替出现。

$$\boxed{\forall h \in \mathsf{Pipe} \times \mathsf{Pipe} \times \mathsf{EventId}.\quad \pi_h(t) \in \mathsf{Pref}(\{\langle S, W \rangle\}^*).}$$

这里 $L^* = \bigcup_{r \geq 0} L^r$，$L^0 = \{\langle\rangle\}$，$L^{r+1} = \{uv \mid u \in L^r, v \in L\}$。每个等待完成对应同一同步键下紧邻的前一个设置完成。初始 flag 未设置，每次等待完成消费一个通知。

A5：等待完成先于目标 pipeline 的后续发起。

$$\boxed{\forall w, u \in \mathsf{Op}(P).\ \forall p, q \in \mathsf{Pipe}.\quad \mathsf{head}(w) = \mathsf{waitflag}(p,q) \wedge w \prec_{q,t} u \Rightarrow D_t(w) \lhd_t S_t(u).}$$

该约束包含其他并发分支向相同目标 pipeline 发起的操作。

3.3 互斥锁

对锁标识 $\ell \in \mathsf{Val}$，定义

$$\begin{aligned} E_\ell(t) = \{e \in \mathsf{ids}(t) \mid \exists o \in \mathsf{Op}(P), s \in S_t(o).\ e \in D_t(o) \wedge \mathsf{pay}(s) = \ell \\ \wedge\, \mathsf{kind}(e) \in \{\mathsf{lock_end}, \mathsf{unlock}\}\}, \end{aligned}$$

$$\lambda_\ell(e) = \begin{cases} A_{\mathsf{thread}(\mathsf{owner}(e))}, & \mathsf{kind}(e) = \mathsf{lock_end}, \\ U_{\mathsf{thread}(\mathsf{owner}(e))}, & \mathsf{kind}(e) = \mathsf{unlock}, \end{cases}$$

$$\pi_\ell(t) = \mathsf{proj}_t(E_\ell(t), \lambda_\ell).$$

A_j, U_j 是分别表示分支 j 成功获取和释放锁的字母。

A6：独占获取与所属分支释放。

$$\boxed{\forall \ell \in \mathsf{Val}.\quad \pi_\ell(t) \in \mathsf{Pref}\left(\left(\bigcup_{j=1}^{m} \{\langle A_j, U_j \rangle\}\right)^*\right).}$$

获取和释放按所属分支配对，持有区间之间互不重叠。锁采用不可重入语义。

4 允许的 Trace

令 $\mathcal{A}_{\mathrm{sync}} = \{\mathrm{A1}, \ldots, \mathrm{A6}\}$。公理满足关系和程序的同步 trace 语义定义为

$$(t, \kappa) \models \mathcal{A}_{\mathrm{sync}} \quad \Longleftrightarrow \quad \bigwedge_{i=1}^{6} \mathrm{A}i(P, t, \kappa),$$

$$\boxed{[\![P]\!]_{\mathrm{sync}} = \{(t, \kappa) \in \mathsf{Cand}(P) \mid (t, \kappa) \models \mathcal{A}_{\mathrm{sync}}\}.}$$

$\mathrm{A}i(P, t, \kappa)$ 表示第 i 项公理在给定程序及候选 trace 上成立。

例如，r 是源 pipeline p 上的读取，f 是随后发起的 setflag$[p, q]$，w 是匹配的 waitflag$[p, q]$，c 是目标 pipeline q 上位于等待之后的计算。对这些操作已出现的单个发起、完成事件，分别用 s_o, d_o 表示。公理给出

$$d_r <_t d_f <_t d_w <_t s_c.$$

源 pipeline 在 s_f 之后发起的另一读取 u 可以具有如下顺序：

$$s_r\ s_f\ s_u\ s_w\ d_r\ d_f\ d_w\ s_c\ d_c\ d_u.$$

这条 trace 中 s_u 先于 d_r, d_f，而 d_f 先于 d_u。